



Introducción a las herramientas NoSQL



¿Qué es No SQL?

A history of databases in No-tation

1970: NoSQL = We have no SQL

1980: NoSQL = Know SQL

2000: NoSQL = No SQL!

2005: NoSQL = Not only SQL

2013: NoSQL = No, SQL!

(R)DB(MS)

SAMSUNG



12:11



¿Qué es NoSQL?

No existe una definición exacta de lo que es un sistema NoSQL. Pero en mayor o menor medida los mismos cumplen ciertas características:

- **No** usan un **modelo relacional** para guardar la información.
 - **No** usan el lenguaje **SQL** para obtener la información.
 - **No fuerzan un esquema** por lo que se le puede agregar campos diferentes a diferentes valores.
 - **Corren en cluster de hardware barato.**
 - **Escalan horizontalmente** simple, por diseño.
 - Eligen nuevas propiedades en vez de las ACID tradicionales.
- 

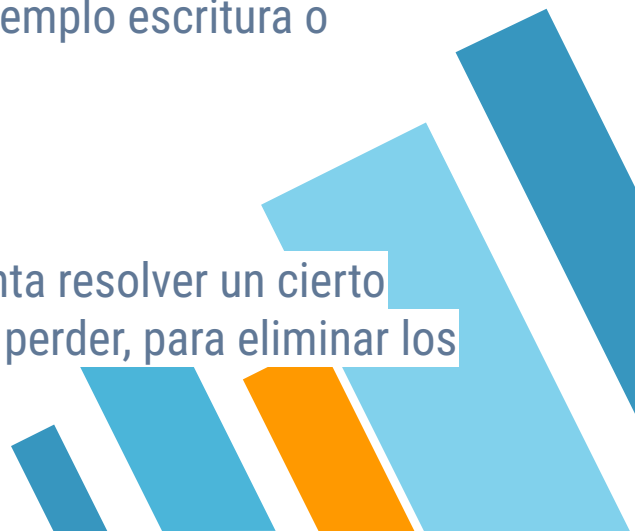


¿Por qué ayuda NoSQL?

Las bases de datos **NoSQL** intentan resolver los problemas de escalabilidad mediante a una o más de las siguientes estrategias:

- **Relajar restricciones** de los esquemas (o directamente los esquemas).
- Quitando o **relajando el cumplimiento de ACID**
- Usando **índices más acorde al tipo de operación** (por ejemplo escritura o modificación).
- **Escalan horizontalmente.**

No todos los stores usan la misma estrategia, cada uno intenta resolver un cierto problema y para ello elige cuales implementar y que se debe perder, para eliminar los problemas de escalabilidad.



¿Cual es el problema de los RDBMS?

- Son **demasiados rígidos** para los nuevos escenarios
- La idea que **es una herramienta para todo** no aplica más.
- No es ideal para guardar **data no estructurada**.
- **Difícil de escalar** y mantener un millones de registros.
- **La estructura de datos** utilizada por estos sistemas están optimizadas para lecturas y utilizando poca cantidad de memoria.





Razones de elección



Brewer's Teorema CAP

- **Consistency:** Indica que todos los nodos devuelven el último dato escrito.
- **Availability:** Garantiza que todo request será inmediatamente respondido (puede ser con error)
- **Partition tolerance:** El sistema sigue funcionando a pesar de que está particionado y alguno de los nodos falla o pierde información.

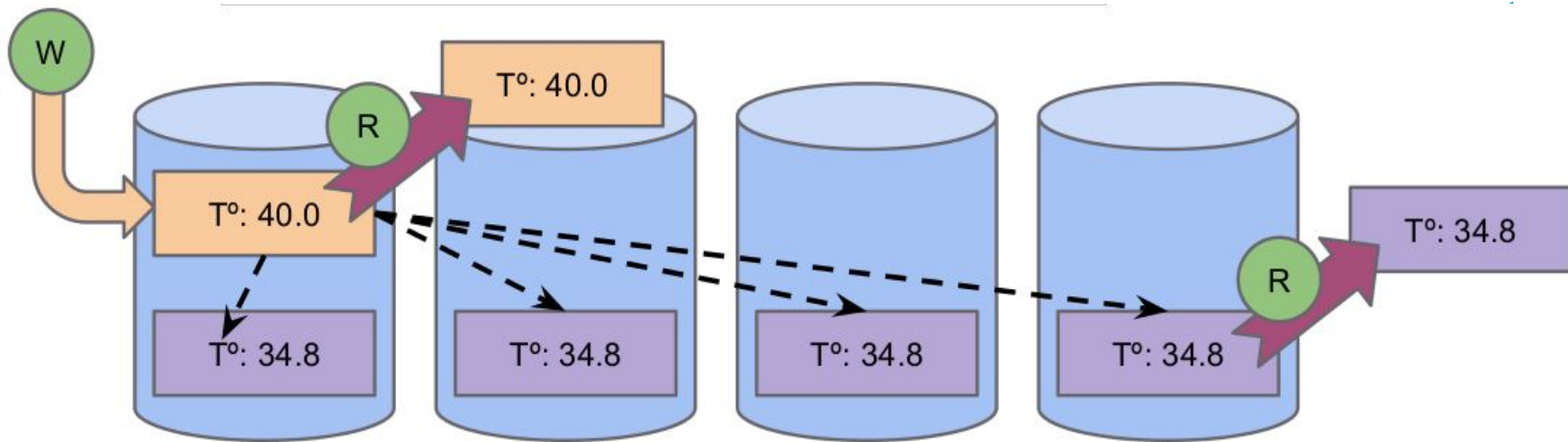
El teorema indica que no se pueden tener las 3, se debe elegir 2. Como estamos hablando de herramientas **que escalan horizontalmente necesitamos partición**. Por lo que hay que elegir entre **Consistencia o Disponibilidad**.

En general (no siempre) las **NoSql** eligen disponibilidad por lo que eligen aplicar consistencia eventual.



Consistencia Eventual

- Promete que dado una escritura, si no hay una nueva actualización todas las réplicas coincidirá eventualmente en el último valor escrito.
- La cantidad de réplicas afecta en la velocidad. Menos réplicas la distribución es más rápida, pero el riesgo de pérdida de datos es mayor.

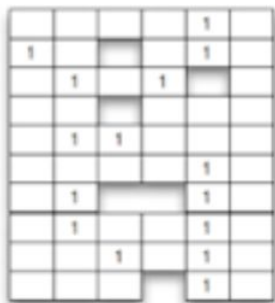


Tipos de Stores NoSQL

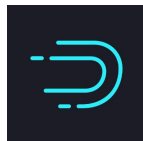
Key-Value



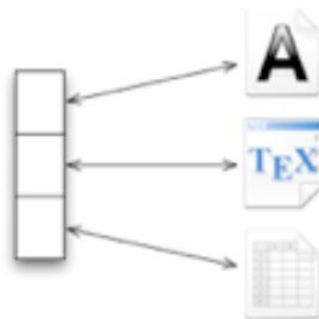
Column



APACHE
HBASE



Document



Graph



[And a few more](#)

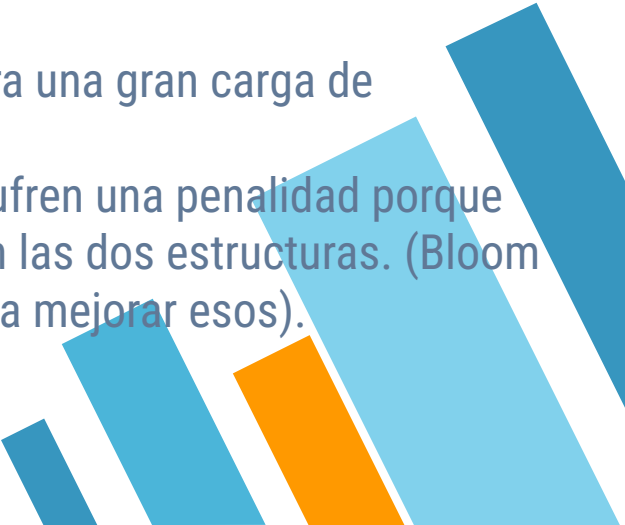


NoSQL: B+ Trees vs LSM Trees

B+ Trees

- Usados como índices para las bases de datos tradicionales.
- Guarda la información de manera que sea eficiente la lectura del filesystem.
- Muchos accesos al disco a la hora de escribir.

LSM Trees (Log-structured merge-tree)

- Índices para bases de estilo Big-Table.
 - Mantiene dos tipos de estructuras separadas, cada uno optimizada para su medio (generalmente uno en memoria otro en disco)
 - Orientados para una gran carga de escritura.
 - Las lecturas sufren una penalidad porque hay que leer en las dos estructuras. (Bloom Filters ayudan a mejorar esos).
- 



Key Value Store



Key Value Stores

- Un store key-value es una base de datos "simple" que usa un Vector asociativo (un mapa o diccionario) como modelo de datos. Los registros se pueden pensar como un par clave - valor.
- **La clave** generalmente **es un string arbitrario** que sirve para identificar al dato (puede ser un nombre de archivo, uri, hash, uuid, etc)
- **El valor** puede ser cualquier cosa, generalmente guardado como un **BLOB** y no requiere ni tiene esquema o modelo de dato.

Al guardarse por clave, **no es necesario indexar** la información para las búsquedas. Por contrapartida no se puede filtrar y/o buscar por elementos del valor, los mismos se consideran "opacos".





Key Value Stores

- En general, key-value stores **no tienen query language**. Proveen operaciones simples para guardar (**put**), borrar (**del**) y obtener el valor de una clave (**get**).
 - Las búsquedas son por clave (`get(key)`), el valor es opaco, no se puede buscar por elementos del valor.
 - Esto permite que no se requieran índices complejos.
 - Tanto que los índices de clave no son estructuras en memoria. Generalmente se "indiza" usando el file system.
 - **Todo esto provoca que sean rápidos para escritura y fácil de usar.**
- 



redis



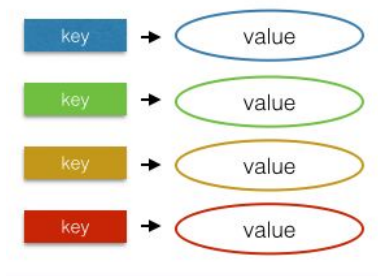
¿Que es Redis?

Redis es **un motor de base de datos en memoria**, basado en el almacenamiento en tablas de hashes (**clave/valor**).



Key Value store

Un key value store guarda elementos que tienen dos componentes.



La inserción, **búsqueda** y borrado son por **key**.
El value es **opaco**.



¿Porqué una base en memoria?

Es más rápido

**Puedo perder la
información**

Puedo activar
persistencia a
disco
Pero...



Redis - Data Types

Redis además de ser key/value permite *darle tipos a los valores* y trabajar con la semántica de esos tipo:

hello world	String
011011010110111101101101	Bitmap
{23334}{6634-720}{916}	Bitfield
{a: "hello", b: "world"}	Hash
[A>B>C>C]	List
{A<B<C}	Set
{A:1, B:2, C:3}	Sorted set
{A: (50.1, 0,5)}	Geospatial
01101101 01101111 01101101	Hyperlog
{id1=time1.seq((a: "foo", a: "bar")) }	Stream



Casos de Uso (Recetas)

- Cachés
 - Sesiones de usuario
 - Carritos de la compra
 - Contadores y estadísticas
 - Locks
 - Estados
- 



Key Value

Para una clave se guarda un valor de tipo string

```
> SET pc:1 Intel  
OK
```

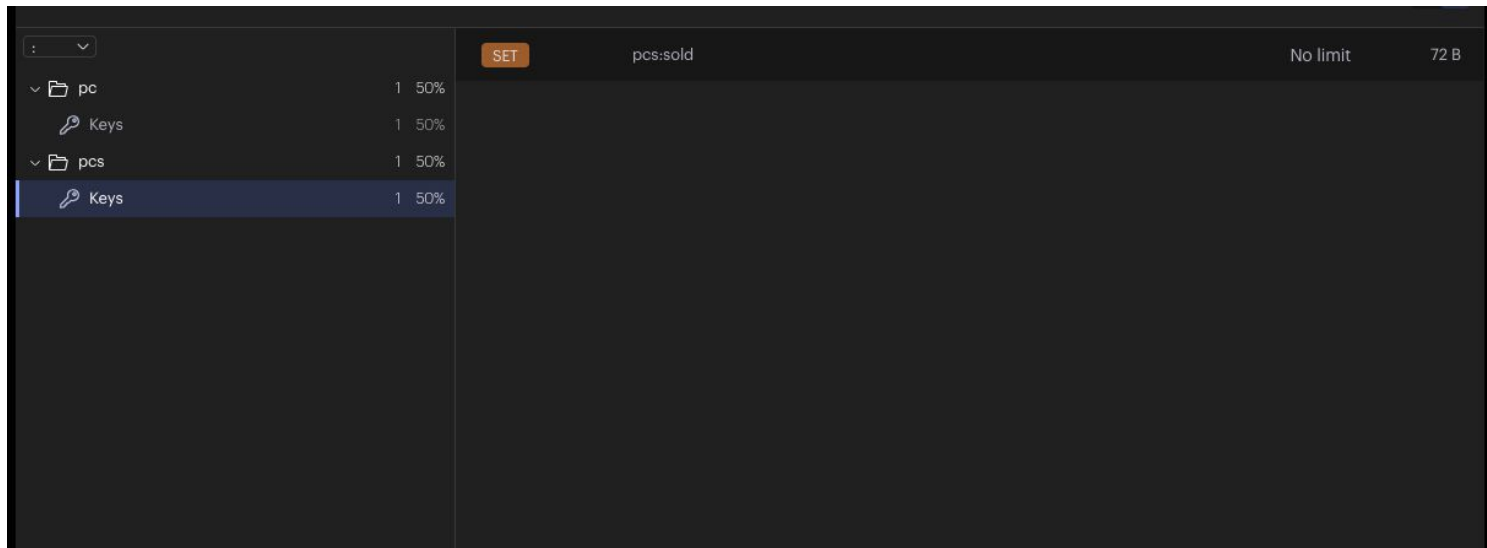
```
> GET pc:1  
"Intel"
```

```
> DEL pc:1  
OK
```



Key Jerarquía

Las claves son strings y son independientes entre sí. Muchas veces se usa "/" o ":" para dar una "sensación" de jerarquía



Keys Modificadores

A las keys se le puede agregar modificadores para controlar sus inserciones

```
> SET pc:1 Intel NX  
OK
```

Si no existe

```
> SET pc:1 Intel XX  
OK / (nil)
```

Si existe

```
> SET pc:1 Intel EX 60  
OK
```

Expira en 60s

Tipos - Contadores

A una key numérica se la puede incrementar

```
> SET pc:1 10
OK

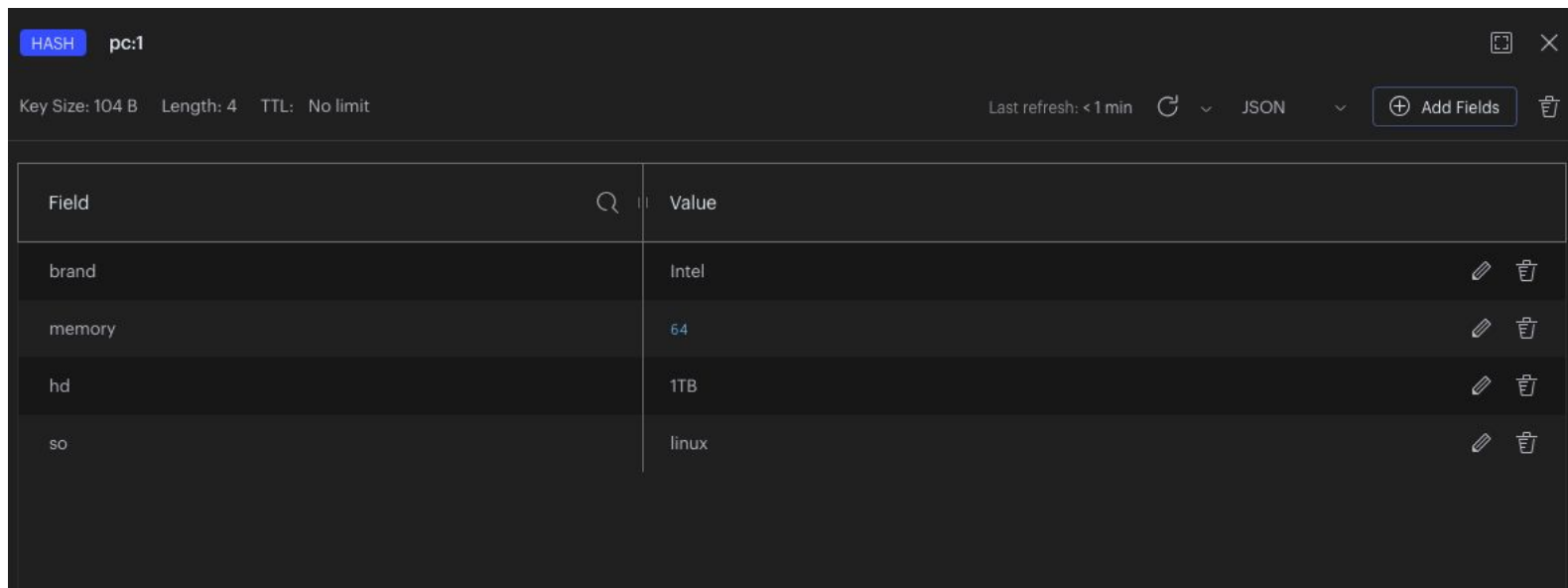
> INCR pc:1
OK

> GET pc:1
11

> INCRBY pc:1 10
OK
```


Tipos - Hashes

Hashes son elementos que representan una colección de pares campo y valor



The screenshot shows a web interface for a Hash database. At the top, there's a header with 'HASH' in a blue box and 'pc:1'. Below this, a status bar shows 'Key Size: 104 B', 'Length: 4', and 'TTL: No limit'. On the right, it says 'Last refresh: < 1 min' with a refresh icon, a dropdown menu set to 'JSON', and a button '+ Add Fields'. The main part of the interface is a table with two columns: 'Field' and 'Value'. The table contains five rows of data. Each row has edit and delete icons on the right. The data is as follows:

Field	Value
brand	Intel
memory	64
hd	1TB
so	linux

Tipos - Hashes

```
> HSET pc:1 brand Intel memory 64 hd 1TB so linux  
      (integer) 4
```

```
> HGET pc:1 brand  
      "Intel"
```

```
> HGET pc:1 memory  
      "64"
```

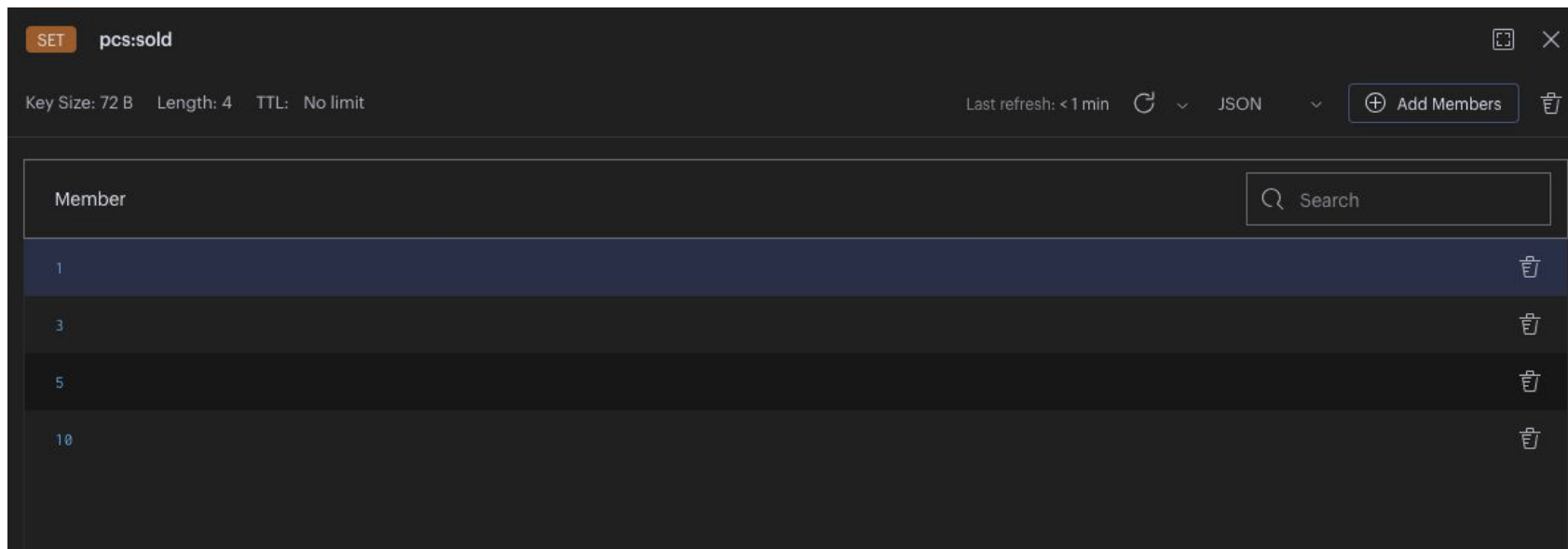
Tipos - Hashes

```
> hgetall pc:1
```

- 1) "brand"
- 2) "Intel"
- 3) "memory"
- 4) "64"
- 5) "hd"
- 6) "1TB"
- 7) "so"
- 8) "linux"

Tipos - Sets

Un set es una colección sin orden sin repetidos



The screenshot shows a Redis key viewer interface for a SET key named 'pcs:sold'. The key has a size of 72 B, a length of 4, and no TTL. The members of the set are 1, 3, 5, and 10. The interface includes a search bar and an 'Add Members' button.

Member
1
3
5
10

Tipos - Sets

```
> SADD pcs:sold 1
```

```
(integer) 1
```

```
> SADD pcs:sold 1
```

```
(integer) 0
```

```
> SADD pcs:available 2 3
```

```
(integer) 2
```

Tipos - Sets

```
> SISMEMBER pcs:sold 1
```

```
(integer) 1
```

```
> SISMEMBER pcs:sold 2
```

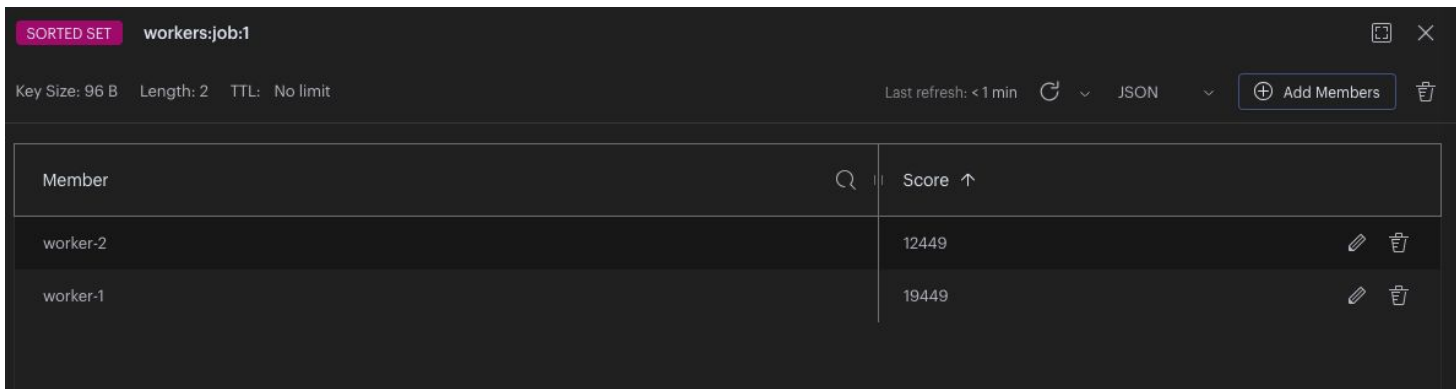
```
(integer) 0
```

```
> SCARD pcs:sold
```

```
(integer) 2
```

Tipos - Sets Ordenados

Es una colección de strings ordenados por un segundo valor



The screenshot shows a Redis Sorted Set named 'workers:job:1'. The interface includes a header with the set name, key size (96 B), length (2), and TTL (No limit). Below the header is a table with two columns: 'Member' and 'Score'. The table lists two members: 'worker-2' with a score of 12449 and 'worker-1' with a score of 19449. The scores are sorted in ascending order. The interface also features a search icon, a refresh button, a dropdown menu for JSON, and an 'Add Members' button.

Member	Score
worker-2	12449
worker-1	19449

Tipos - Sets Ordenados

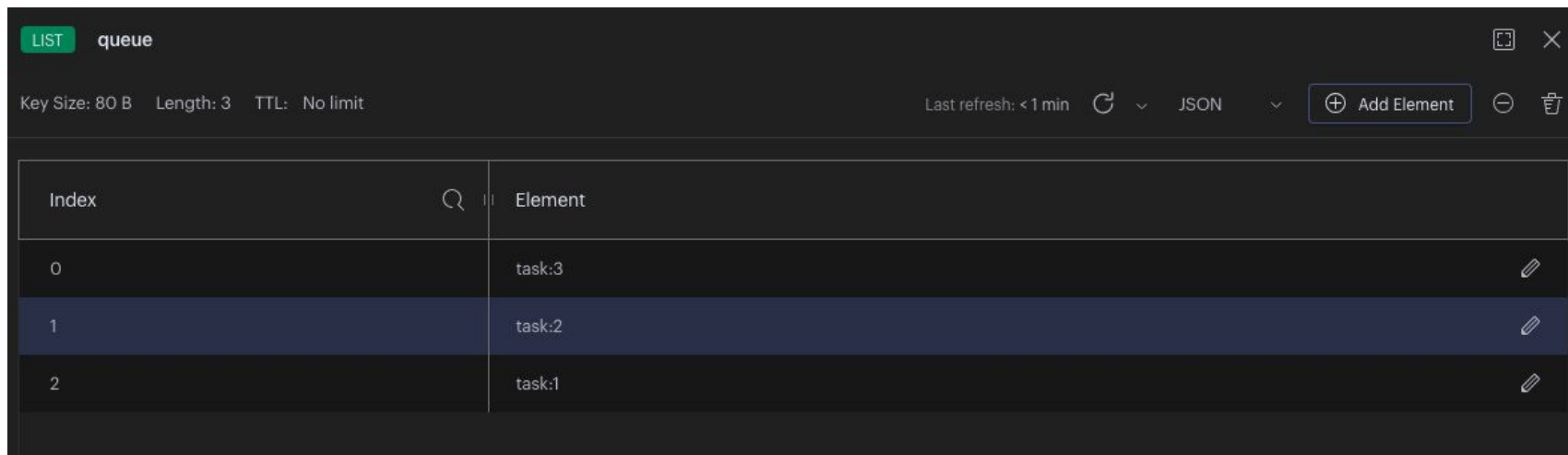
```
> zadd workers:job:1 19449 "worker-1"
(integer) 1

> zadd workers:job:1 12449 "worker-2"
(integer) 1

> ZRANGEBYSCORE workers:job:1 0 inf WITHSCORES
1) "worker-2"
2) "12449"
3) "worker-1"
4) "19449"
```


Tipos - Listas

Una linked list de valores string.



The screenshot shows a Redis Queue interface. At the top, there's a header with 'LIST' in a green box and 'queue'. Below this, metadata is displayed: 'Key Size: 80 B', 'Length: 3', and 'TTL: No limit'. On the right side of the header, there's a 'Last refresh: < 1 min' indicator, a refresh button, a format dropdown set to 'JSON', and an 'Add Element' button. The main area contains a table with two columns: 'Index' and 'Element'. The table lists three elements: 'task:3' at index 0, 'task:2' at index 1 (which is highlighted), and 'task:1' at index 2. Each row has a small edit icon on the right.

Index	Element
0	task:3
1	task:2
2	task:1

Tipos - Listas

```
> lpush queue task:1  
      (integer) 1
```

```
> lpush queue task:2  
      (integer) 1
```

```
> lpush queue task:3  
      (integer) 1
```

```
> rpop queue  
      "task:1"
```



Pub/Sub

Aunque no es un "tipo" per/se Redis provee canales sobre los cuales se puede implementar comunicación pub/sub.

```
> SUBSCRIBE changes:tasks changes:pcs  
...  
  
> PUBLISH changes:pcs pc:1  
  
> UNSUBSCRIBE changes:pcs
```





Pub/Sub

- El cliente se puede subscribir a múltiples canales (se pueden usar patterns).
 - Para el subscriber bloquea una conexión a redis (para todas sus suscripciones)
 - Pub/Sub es fire and forget.
 - Semántica at-most-once.
 - No comparte el keyspace con el key/value store
 - Para una semántica no tan robusta se puede usar Streams
- 



REDIS: references

- <http://redis.io/topics/introduction>
 - <https://redis.io/topics/quickstart>
 - <https://try.redis.io/>
- 



Columnar Storage

- Una base de datos columnar **está optimizada para leer columnas de la información** (en contrapartida de leer filas).
 - El principal uso es para hacer queries analíticas, que se ven beneficiadas por este tipo de guardado.
 - Reducen mucho los requerimientos de **accesos de I/O y la cantidad de data** a levantar.
 - Permiten una mejor compresión, ya que la data por archivos es más homogénea.
 - **I/O** se reduce ya que solo se leen las columnas requeridas para resolver la respuesta (en vez de realizar un "full scan").
 - Se diseñan fácil para escalar horizontalmente en hardware barato.
- 

Columnar Storage

Aquí un ejemplo de 3 filas con 3 columnas.

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

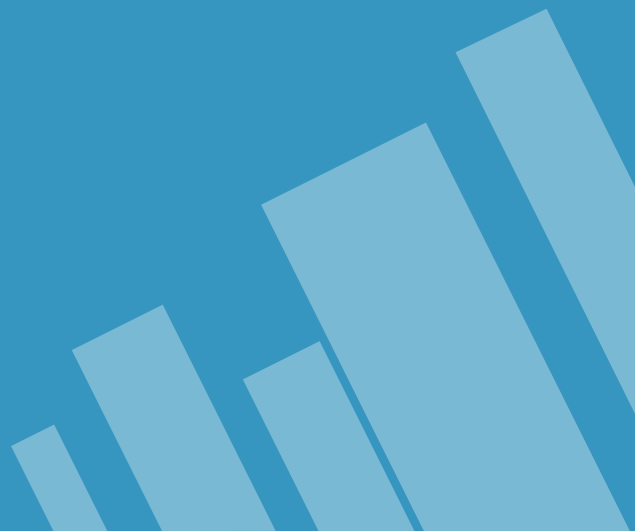
En una base de "datos de filas" se guarda (conceptualmente) así:

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

En cambio en una base columnar, se guarda así:

A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

y probablemente cada columna se escribe en un archivo o directorio separado



Columnar Store



APACHE
HBASE





HBASE

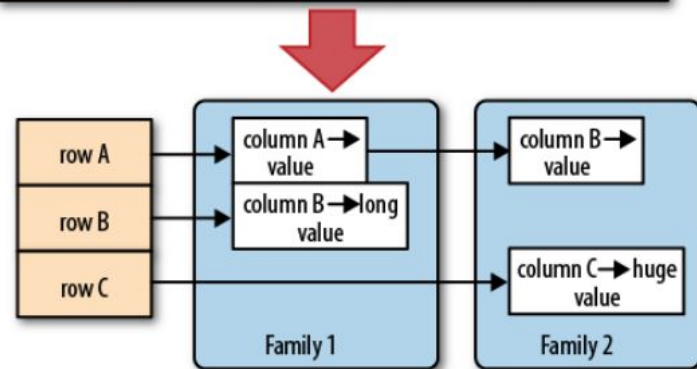
Apache HBase es una base de datos no relacional, columnar, distribuida, escalable, consistente, con baja latencia y de random access, construida sobre la tecnología de Apache Hadoop;

- Basado en el paper " BigTable de Google"
 - Mapa múltiple, persistente distribuido.
 - Provee funcionalidad NoSQL sobre Hadoop.
- 

HBase: Data Model

- **Tables** son mapas de filas.
- **Cada fila tiene un primary key** que permite encontrar los elementos en la tabla.
- Cada fila tiene un conjunto de "**familia de columnas**" que se especifica como parte del esquema.
- **Se pueden agregar columnas dinámicamente** (no se definen columnas)
- **Las celdas** (combinación de fila y columna) son `byte[]` y con un timestamp. NULLs no ocupan espacio.

	column A (int)	column B (varchar)	column C (boolean)	column D (date)
row A				
row B				
row C			NULL?	
row D				



Hbase Ejemplos de uso

```
$> create 'empl', 'personal data', 'professional data'
$> put 'empl','1','personal data:name','mark'
$> put 'empl','1','personal data:city','ohio'
$> put 'empl','1','professional data:designation','manager'
$> put 'empl','1','professional data:salary','50000'
$> put 'empl','2','personal data:name','john'
$> put 'empl','2','personal data:city','nyc'
$> put 'empl','2','professional data:designation','typist'
$> put 'empl','2','professional data:salary','40000'
$> get 'empl', '1', 'personal data:name'
$> delete 'empl', '2', 'professional data:designation'
$> delete 'empl', '1'
```



HBase Components

- Las tablas se componen de una o más regiones
 - **RegionServer:**
 - Pueden tener múltiples regiones pero solo se sirven de a una por vez.
 - Las regiones están "co-locadas" con los data nodes de HDFS para mantener la "data locality."
 - Cuando se escribe información en una región, tres elementos participan de la escritura.
 - **HLog:** un **Write Ahead Log** que guarda las operaciones de manera secuencial e inmutable.
 - **MemStore:** un LSM-Tree en memoria que guarda las últimas inserciones.
 - **HFiles**
 - Un archivo que se genera cuando el Memstore excede de cierto tamaño para flushear a disco. También son append e inmutables
 - Si la cantidad de HFiles crece para una región más allá de un límite se lanza un proceso de compactación que reduce los archivos y su tamaño.
- 



HBase: read and delete

Cuando un cliente pide un registro a HBASE el proceso es:

- Buscar en el MemStore.
- De no encontrarlo se busca en el Region Server's Block cache (un LRU cache de filas).
- Si no lo encuentra se levantan los HFILEs hasta encontrar el registro

El -proceso de borrado, escribe una operación de "**tombstone**" en el MemStore. De manera que al buscar el registro se encuentra la operación de borrado y se devuelve el no existe.

Cuando se ejecuta una compaction de los archivos es que se borra permanentemente los registros que fueron borrados lógicamente vía tombstones.



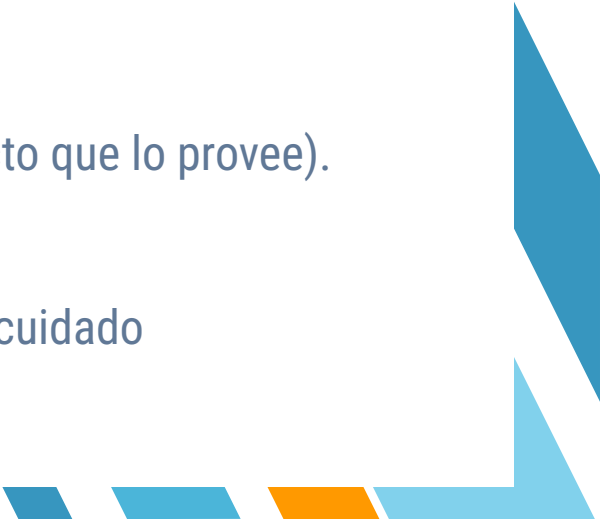


HBase Summary

Ventajas

- Reads/writes **consistentes**.
- Automatic **sharding**.
- Automatic **RegionServer failover and HTable compaction**.
- Integración con HDFS y soporte para MapReduce .
- Java Client y Thrift/REST APIs.

Desventajas

- No provee soporte de **SQL** (Apache Phoenix es un proyecto que lo provee).
 - No tiene **transacciones**.
 - **Requiere** un cluster de **Hadoop** y uno de **Zookeeper**.
 - Tiene **Tendencia a tener "hot spots"** si no se maneja con cuidado
- 



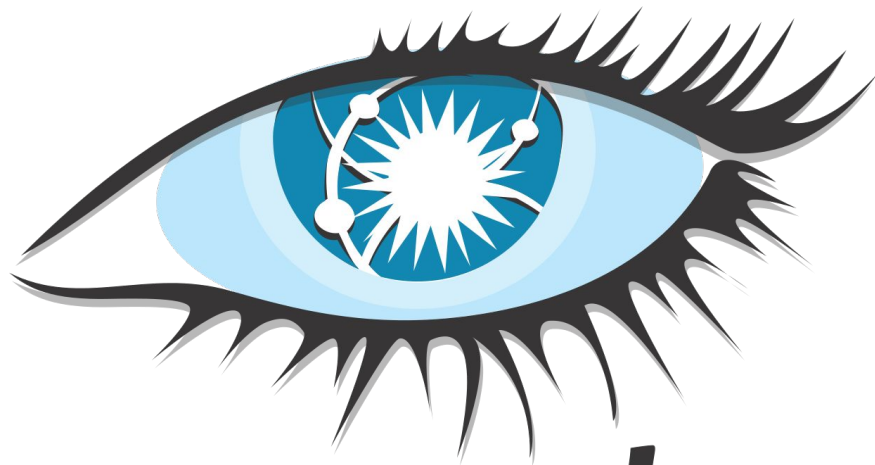
HBase: references

Book

- “HBase: The Definitive Guide”, Lars George, First Edition, 2011.

Online

- <http://hbase.apache.org/book.html>
 - <http://www.tutorialspoint.com/hbase>
 - <http://www.dbms2.com/2015/03/10/notes-on-hbase/>
- 



cassandra

What is NoSQL?

- » Desarrollado en Facebook
- » Modelo de datos basado en el paper **Google Big Table**.
- » Características de distribución basadas en el paper **Amazon's DynamoDB** paper.
- » Top level Apache project desde 2010.

Cassandra - A Decentralized Structured Storage System

Avinash Lakshman
Facebook

Prashant Malik
Facebook

ABSTRACT

Cassandra is a distributed storage system for managing very large amounts of structured data spread out across many commodity servers, while providing highly available service with no single point of failure. Cassandra aims to run on top of an infrastructure of hundreds of nodes (possibly spread across different data centers). At this scale, small and large components fail continuously. The way Cassandra manages the persistent state in the face of these failures drives the reliability and scalability of the software systems relying on this service. While in many ways Cassandra resembles a database and shares many design and implementation strategies therewith, Cassandra does not support a full relational data model; instead, it provides clients with a simple data model that supports dynamic control over data layout and format. Cassandra system was designed to run on cheap commodity hardware and handle high write throughput while not sacrificing read efficiency.

1. INTRODUCTION

Facebook runs the largest social networking platform that serves hundreds of millions of users at peak times using tens of thousands of servers located in many data centers around the world. There are strict operational requirements on Facebook's platform in terms of performance, reliability and efficiency, and to support *continuous growth* the platform needs to be highly scalable. Dealing with failures in an infrastructure comprised of thousands of components is our standard mode of operation; there are always a small but significant number of server and network components that are failing at any given time. As such, the software systems need to be constructed in a manner that treats failures as the norm rather than the exception. To meet the reliability and scalability needs described above Facebook has developed Cassandra.

Cassandra uses a synthesis of well known techniques to achieve scalability and availability. Cassandra was designed to fulfill the storage needs of the Inbox Search problem. In-

box Search is a feature that enables users to search through their Facebook Inbox. At Facebook this meant the system was required to handle a very high write throughput, billions of writes per day, and also scale with the number of users. Since users are served from data centers that are geographically distributed, being able to replicate data across data centers was key to keep search latencies down. Inbox Search was launched in June of 2008 for around 100 million users and today we are at over 250 million users and Cassandra has kept up the promise so far. Cassandra is now deployed as the backend storage system for multiple services within Facebook.

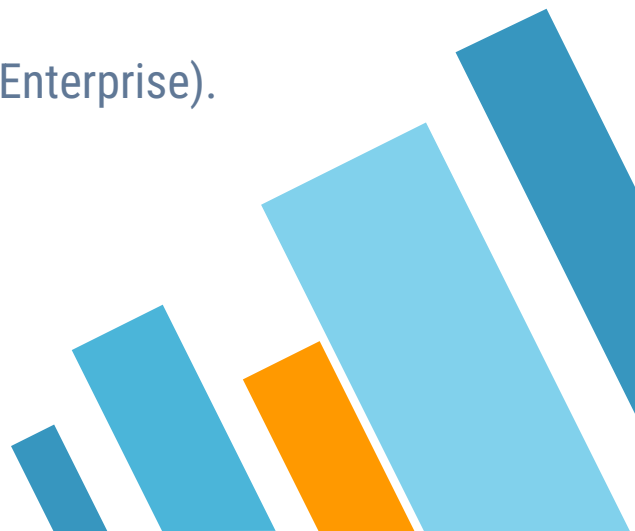
This paper is structured as follows. Section 2 talks about related work, some of which has been very influential on our design. Section 3 presents the data model in more detail. Section 4 presents the overview of the client API. Section 5 presents the system design and the distributed algorithms that make Cassandra work. Section 6 details the experiences of making Cassandra work and refinements to improve performance. In Section 6.1 we describe how one of the applications in the Facebook platform uses Cassandra. Finally Section 7 concludes with future work on Cassandra.

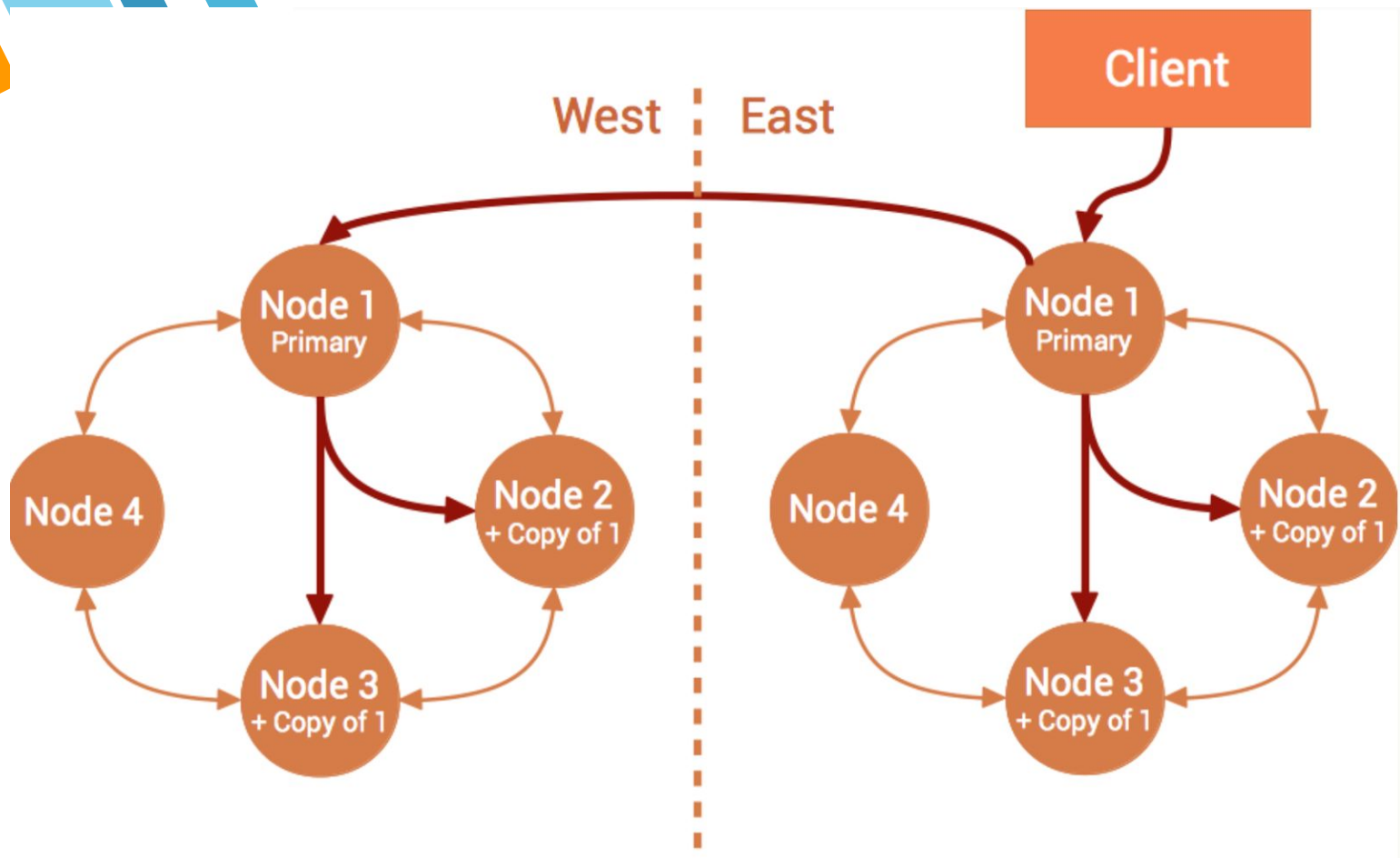
2. RELATED WORK

Distributing data for performance, availability and durability has been widely studied in the file system and database communities. Compared to P2P storage systems that only support flat namespaces, distributed file systems typically support hierarchical namespaces. Systems like Ficus[14] and Coda[16] replicate files for high availability at the expense of consistency. Update conflicts are typically managed using specialized conflict resolution procedures. Farsite[2] is a distributed file system that does not use any centralized server. Farsite achieves high availability and scalability using replication. The Google File System (GFS)[9] is another distributed file system built for hosting the state of Google's internal applications. GFS uses a simple design with a single master server for hosting the entire metadata and where the data is split into chunks and stored in chunk servers. However the GFS master is now made fault tolerant using the Chubby[3] abstraction. Bayou[18] is a distributed relational database system that allows disconnected operations and provides eventual data consistency. Among these systems, Bayou, Coda and Ficus allow disconnected operations and are resilient to issues such as network partitions and outages. These systems differ on their conflict resolution procedures. For instance, Coda and Ficus perform system level conflict resolution and Bayou allows application level

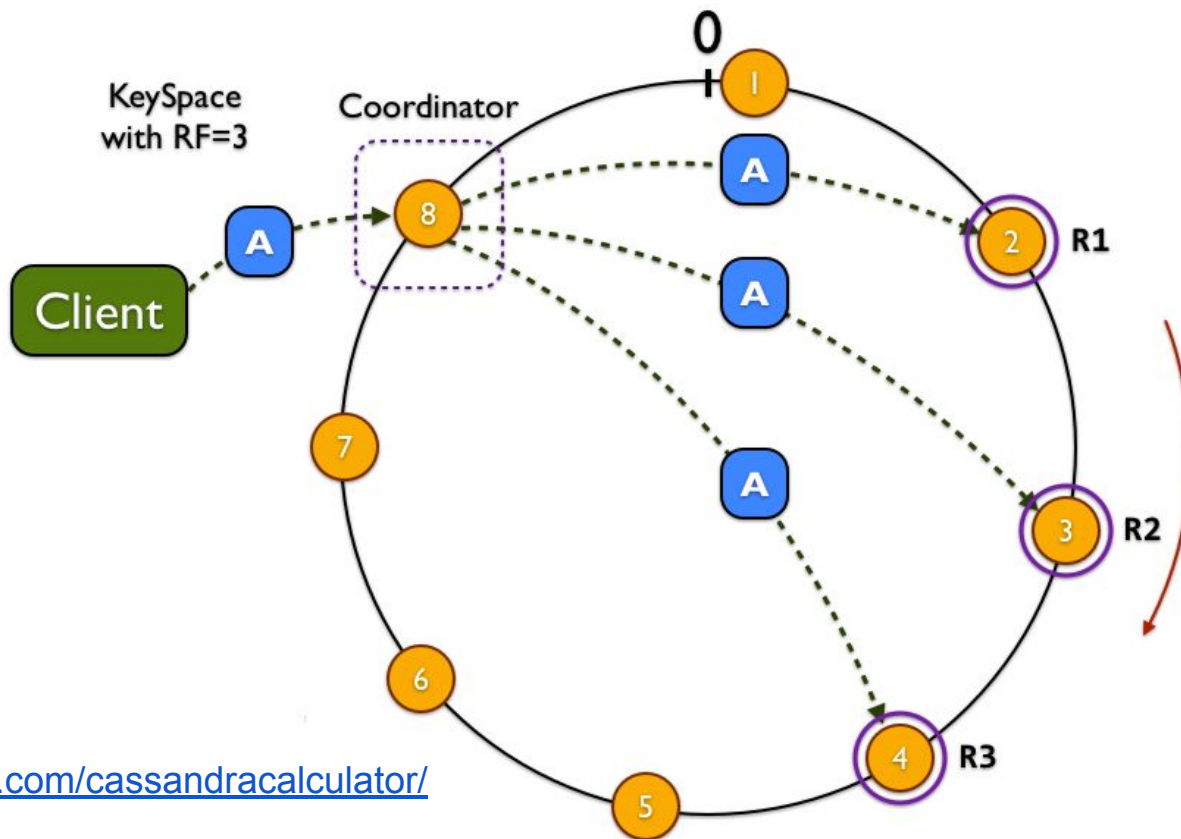


Cassandra: Key Features

- **Decentralized:** cada nodo en el cluster tiene el mismo rol (NO SPOF)
 - **Linear scalability:** La capacidad de lectura y escritura crece lineal con la cantidad de nodos
 - **Tunable consistency:** Los niveles de consistencia son configurable.
 - **Multi datacenter replication.**
 - **Commercial support:** provisto Datastax (DSE: Datastax Enterprise).
- 



Cassandra: Data Partitioning



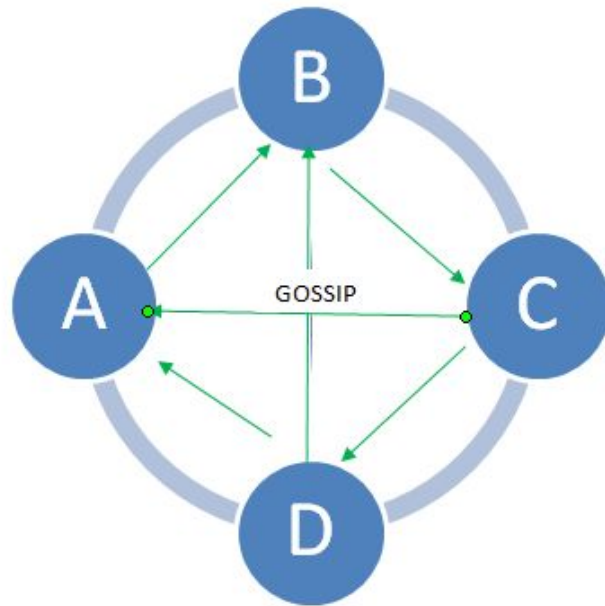


Cassandra: consistency

- **ALL:** todas las réplicas deben dar su ACK
 - **QUORUM:** > 51% de las réplicas deben dar su ACK.
 - **LOCAL_QUORUM:** > 51% de las réplicas en el DC deben dar su ACK.
 - **EACH_QUORUM:** > 51% de las réplicas on ALL DC deben dar su ACK.
 - **ONE:** alcanza con un ACK.
 - **TWO:** alcanza con dos ACKs.
 - **THREE:** tres réplicas deben dar ACK.
 - **LOCAL_ONE:** alcanza con un ACK dentro del DC.
- 

Cassandra: Gossip Protocol

- Los Nodos periódicamente intercambian información de estado tanto propio como lo que "saben" de otros.
- Corre una vez por segundo y cada nodo se comunica con hasta 3 nodos para compartir la información.
- Los mensajes tiene versiones para que se pueda determinar quién tiene "la última"



CQL = Cassandra Query Language

```
CREATE KEYSPACE devices WITH replication =  
{ 'class' : 'SimpleStrategy', 'replication_factor' : 3};  
  
USE devices;
```

```
CREATE TABLE device_events(  
deviceID int,  
TIME int,  
event_count int  
PRIMARY KEY(deviceID, TIME)  
);
```

```
SELECT event_count  
FROM device_events  
WHERE device_id = 1  
AND TIME > '2011-02-03'  
AND TIME <= '2012-01-01';
```

No hay soporte para: JOINS, LIKE, Subqueries, Aggregations.

Los campos en el WHERE son solo los de la Primary Key

(unless ALLOW FILTERING is specified)



Cassandra: Modelo de datos

Debido a la forma de guardado y las restricciones de consulta los creadores de Cassandra recomiendan que el modelo de datos sea definido por "consultas".

Esto significa que **cada consulta tiene una tabla asociada** a pesar de que eso implique **duplicar la información**.

Cassandra Data Modeling Principles

- Know your data
- Know your queries
- Nest data
- Duplicate data





Cassandra: Modelo de datos

Desnormalización

Esto se debe a que en el lenguaje de consulta en el where solo se puede usar los campos de la primary.

Por lo cual cada consulta tiene una tabla con la primary dependiente de que campos pondré en el where.

```
CREATE TABLE comments_by_video (  
    video_title text,  
    comment_id timeuuid,  
    user_id text,  
    video_id timeuuid,  
    comment text,  
    PRIMARY KEY ((video_title), comment_id)  
);  
  
CREATE TABLE comments_by_user (  
    user_login text,  
    comment_id timeuuid,  
    user_id text,  
    video_id timeuuid,  
    comment text,  
    PRIMARY KEY ((user_login), comment_id)
```



Cassandra: Primary Keys

La primary key, se compone de una o dos partes:

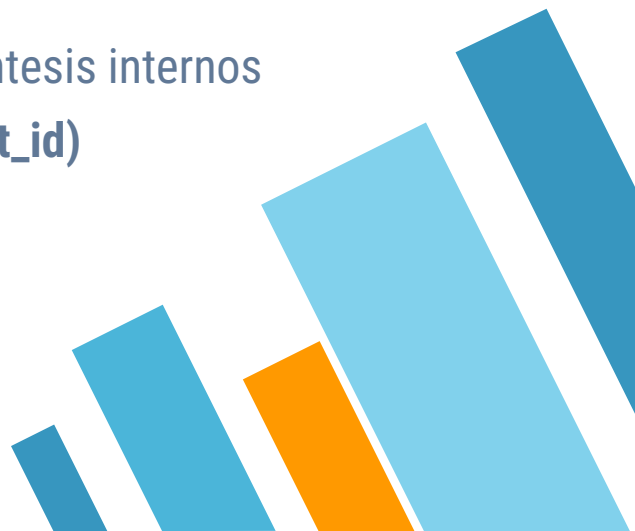
- partition key
- clustering column

Ambas pueden ser compuestas.

Dentro de la estructura de una primary se separan por los paréntesis internos

PRIMARY KEY((video_title), comment_id)

Donde

- video_title: es la partition key
 - comment_id: es la clustering column
- 



Cassandra: Partition Keys

La partition key es la que define quien es el nodo que será "dueño" de la partición y también quienes tienen las réplicas.

Además Cassandra subdivide la información de la tabla en el disco de cada nodo utilizando la estructura de la Primary Key. Funcionando como un "índice físico"

Por esta razón es que las queries **deben tener** a los elementos de la primary en el where (preferente por igualdad).

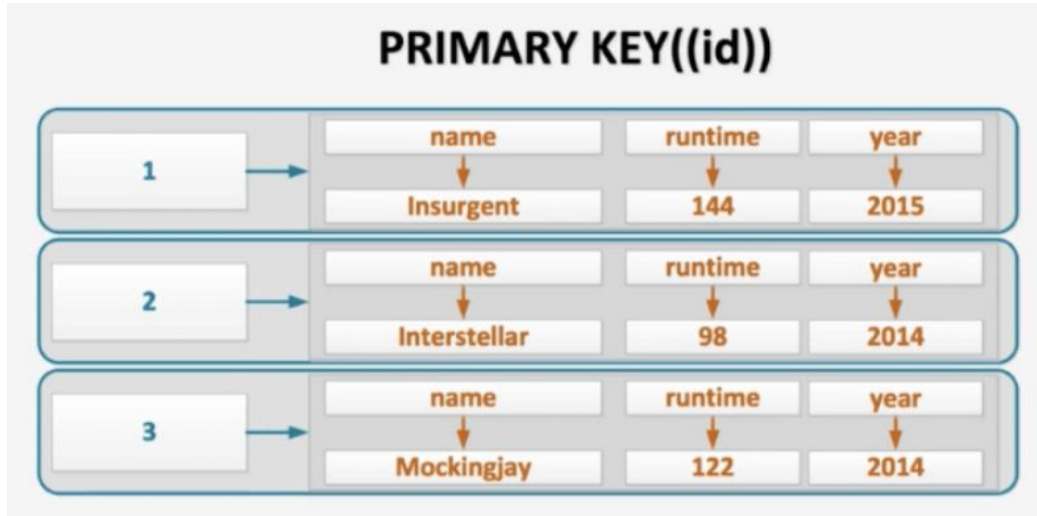


Cassandra: Partition Keys

La partition key es la que define quien es el nodo que será "dueño" de la partición y también quienes tienen las réplicas.

Permite encontrar el registro fácilmente en el cluster.

Debido a esto las queries siempre deben tener a las columnas de la partition key en el where (buscando por igualdad).

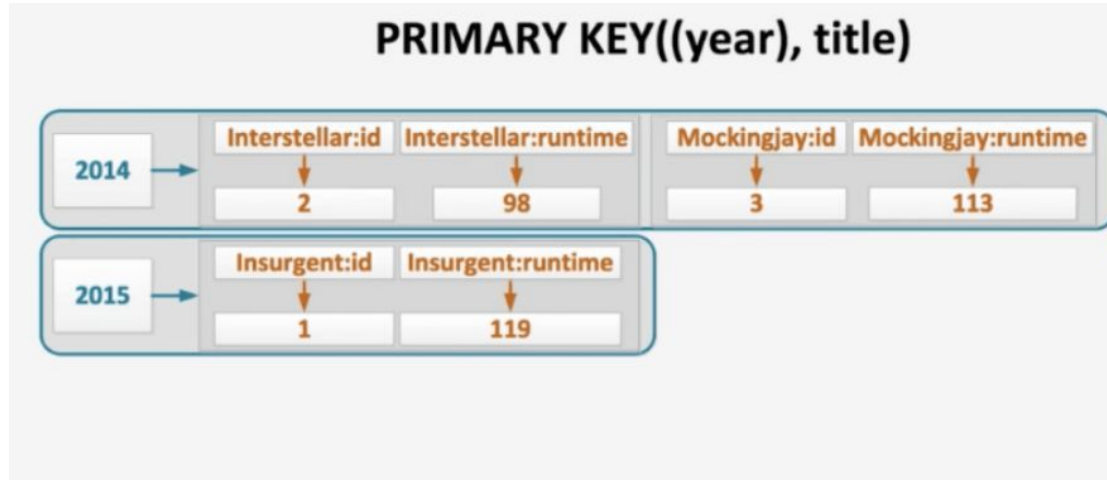


Cassandra: Clustering Columns

Proveen un segundo nivel de direccionamiento en el filesystem.

Se utilizan en el where para hacer consultas de rangos (> y <) y para ordenamientos.

No son obligatorios en la query, pero de usarlos se deben usar en el orden que se definieron en la creación de la tabla.





Cassandra: references

- <https://cassandra.apache.org/doc/latest/>
 - <https://academy.datastax.com/>
 - <http://www.slideshare.net/patrickmcfadin/introduction-to-cassandra-2014>
 - <http://www.slideshare.net/jaykumarpatel/cassandra-data-modeling-best-practices>
- 



Document Oriented Store



Document Oriented Store

- Key Value store, donde **el valor es un documento** (objeto complejo).
 - La mayor diferencia con un kv-store es que el valor no es opaco, por lo que se puede filtrar por y actualizar componentes del valor.
 - Los documentos **no deben cumplir con un esquema**.
 - Lo que los hace perfecto para **información semi estructurada**./.
- 



Beneficios

- **Los documentos son unidades independientes y completas.** La data completa está contigua y desnormalizada en el disco lo que hace fácil de leer.
 - **La lógica de la aplicación es más fácil de codificar.** No es necesario transformar objetos en tablas.
 - **La data no estructurada se escribe fácil.**
- 





MongoDB es una base de datos open source que provee alta performance, alta disponibilidad y escalamiento automático.





MongoDB - Key Features

- **High Performance:** MongoDB provee persistencia de datos con alta performance
 - **Rich Query Language:** Para operaciones CRUD, Query, Data Aggregation, Text Search y Geospatial Queries.
 - **High Availability:** Las replica set proveen: automatic failover y data redundancy.
 - **Horizontal Scalability:** Escalabilidad horizontal viene incorporada como parte de su funcionalidad core
 - **Multiple storage engines, como:**
 - WiredTiger Storage Engine
 - MMAPv1 Storage Engine.
- 

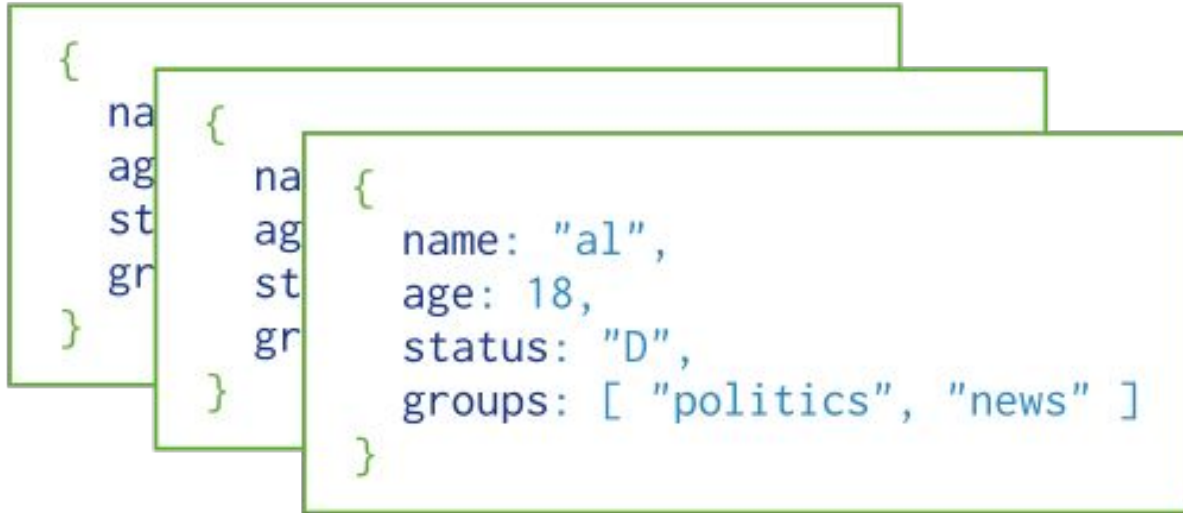
MongoDB Documentos

En MongoDB los documento son un conjunto de pares key values, que se presentan como un JSON (en realidad es Bson una implementación de Json de Mongo):

```
var mydoc = {  
  _id: ObjectId("5099803df3f4948bd2f98391"),  
  name: { first: "Alan", last: "Turing" },  
  birth: new Date('Jun 23, 1912'),  
  death: new Date('Jun 07, 1954'),  
  contribs: [ "Turing machine", "Turing test", "Turingery" ],  
  views : NumberLong(1250000)  
}
```

MongoDB Colecciones y Bases de datos

MongoDB guarda los documentos BSON en colecciones (similares a tablas en RDBMS). Las colecciones se agrupan en "bases de datos"



Collection

MongoDB Commands

Las operaciones de CRUD son las siguientes

<u>db.collection.insert()</u>	Inserta un nuevo documento en la colección.
<u>db.collection.update()</u>	Actualiza un documento de la colección
<u>db.collection.remove()</u>	Borra un documento de la colección.
<u>db.collection.drop()</u>	Borra la colección
<u>db.collection.createIndex()</u>	Crea un índice sobre la colección.
db.dropDatabase()	borra la base de datos

MongoDB crud

```
db.inventory.insertOne(  
  {  
    item: "canvas", qty: 100, tags: ["cotton"],  
    size: { h: 28, w: 35.5, uom: "cm" } }  
)
```

```
db.inventory.updateOne(  
  { item: "paper" },  
  {  
    $set: { "size.uom": "cm", status: "P" },  
    $currentDate: { lastModified: true }  
  }  
)
```

```
db.inventory.deleteMany({ status : "A" })
```



MongoDB Commands

La operación de búsqueda recibe un documento que describe el criterio para matchear.

<code>coll = db.<collection></code>	Genera un alias para la colección
<code>db.collection.find()</code>	Retorna un cursor con todos los documentos en la colección



MongoDB Query

```
db.inventory.find( { status: "A", qty: { $lt: 30 } } )
```

```
db.inventory.find( { $or: [ { status: "A" }, { qty: { $lt: 30 } } ] } )
```

```
db.inventory.find( {  
  status: "A",  
  $or: [ { qty: { $lt: 30 } }, { item: /^p/ } ]  
} )
```

MongoDB Aggregations

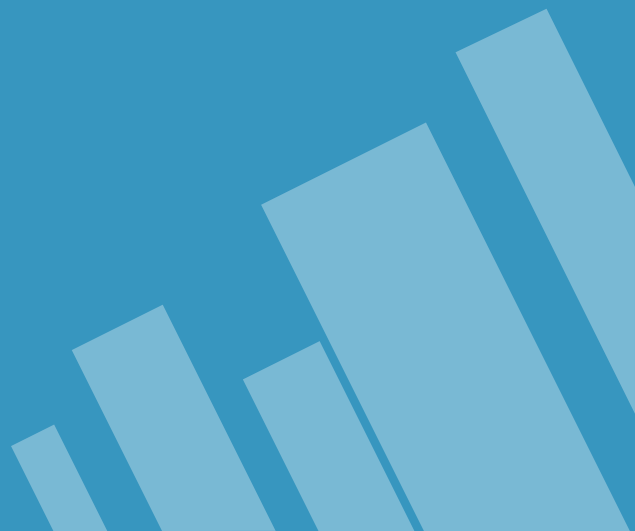
La operación de agregación recibe un pipeline de operaciones que se ejecutan en orden.

```
db.accounts.aggregate([
  {$match : {"index": {$exists: false}}},
  {$project: {_id:0, gender: 1, decade:{ $trunc: {$divide: [ "$age", 10 ]
}}}},
  {$group:{_id: { "gender": "$gender", "decade": "$decade"}, total: {$sum:
1} } }
])
```



References

- » [Manual](#)
 - » [MongoDB University](#)
 - » [MongoDB Blog](#)
- 



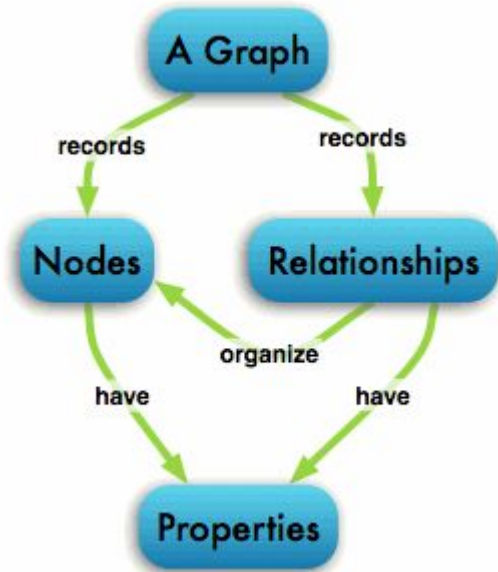
Graph Store



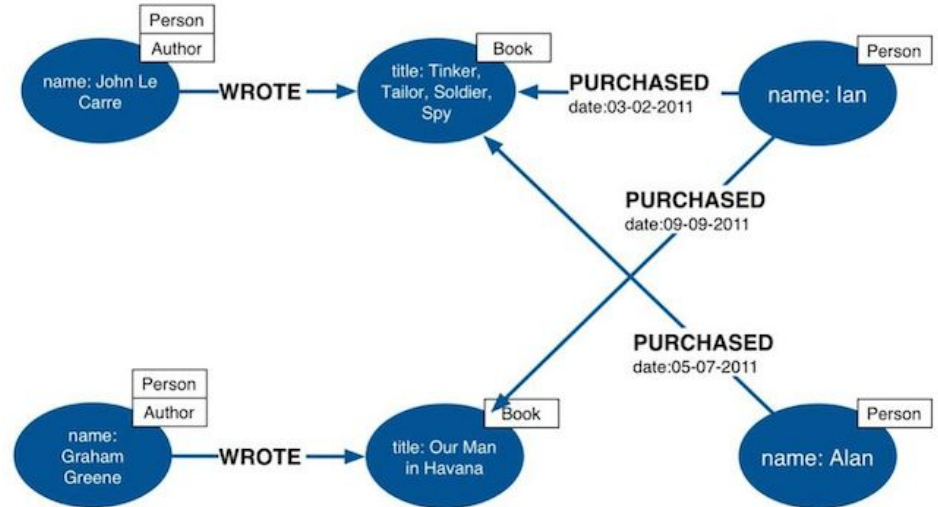
Graph Databases

- Una base de datos de grafos guarda su información como una colección de nodos y aristas.
 - Cada nodo representa una entidad se define mediante a una clave única, un conjunto de aristas de entrada, otro de arista de salida y las propiedades de la entidad (representados por key-value pairs).
 - Cada arista se define por una clave única, su nodo de origen y nodos de destino y un conjunto de propiedades propias (también key-value).
- 

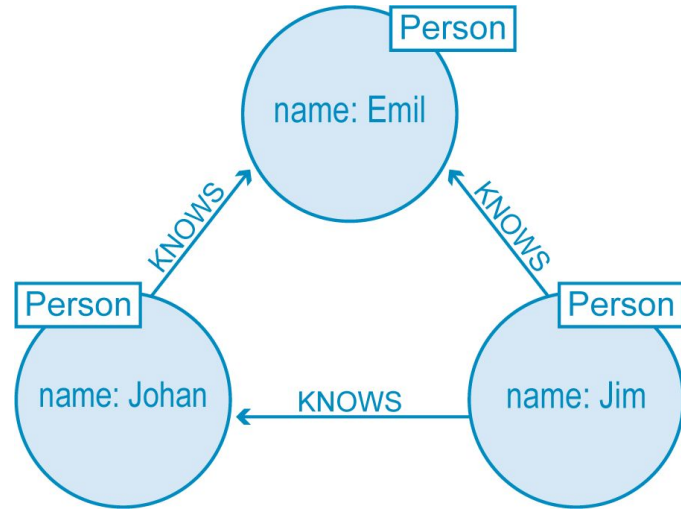
Graph Databases



Labeled Property Graph Data Model



Graph Databases



```
MATCH (a:Person {name:'Jim'})-[:KNOWS]->(b)-[:KNOWS]->(c),  
      (a)-[:KNOWS]->(c)  
RETURN b, c
```



elastic

Solr 

Search As No SQL



Search as NoSQL

- Usan **Apache Lucene** (created by Doug Cutting) como core y estructura de datos. Proveen una API un poco más intuitiva para la búsqueda de información basada en índices de búsqueda invertidos.
 - Los productos más importantes en el mercado son **Elasticsearch and SolrCloud**.
 - Elasticsearch se creó para que **sea altamente escalable** y tenga **alta disponibilidad**
 - Elasticsearch provee un api REST y acceso directo a su API para escribir desde frameworks de procesamiento (Como Hadoop o Spark)
 - Tiene esquemas flexibles para la información (usa Json como soporte para los documentos dentro de sus índices).
- 



elastic

Elastic Search



- Elasticsearch es una engine altamente escalable y open source para realizar full-text search y queries de agregaciones analíticas.
- Permite guardar, buscar y analizar grandes cantidades de información en tiempos cercanos a "real time"
- El engine de búsqueda por texto se basa en Lucene
- Por lo que es una base
 - distribuida
 - multitenant
 - con una interfaz HTTP (Rest)
 - Que guarda documentos JSON.



Definir el mapping de un índice:

PUT /shakespeare

```
{
  "mappings" : {
    "_default_" : {
      "properties" : {
        "speaker" : { "type": "keyword" },
        "play_name" : { "type": "keyword" },
        "line_id" : { "type" : "integer" },
        "speech_number" : { "type" : "integer" }
      }
    }
  }
}
```

Query: buscando los valores que cumplan con las condiciones.

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "age": "40" } }
      ],
      "must_not": [
        { "match": { "state": "ID" } }
      ]
    }
  }
}
```

Query de agregación (múltiple)

```
{ "size": 0,  
  "aggs": {  
    "group_by_state": {  
      "terms": {  
        "field": "state.keyword"  
      },  
      "aggs": {  
        "average_balance": {  
          "avg": {  
            "field": "balance"  
          }  
        }  
      }  
    }  
  }  
}
```


Key Take Aways

- Las bases NoSQL inicialmente se pensaron para soportar volúmenes de datos operacionales. Ahora se las piensa para grandes volúmenes analíticos
- La estructura como se guarda la información en NoSql difiere de SQL y depende mucho del tipo de queries que se quiera hacer
- **Las bases NoSQL son "más simples"** en cuanto a funcionalidad lo que las hace más fácil de escalar
- Con la moda Big Data el **one-size fits all of RDBMs no es más válido.**
- Se dice que estamos en la era de la **persistencia políglota** ya que en vez de usar una base de datos un sistema puede tener varios para cada caso de uso específico.
- Al aparecer múltiples stores en una empresa se requiere una capa de abstracción lógica que algunos denominan **Logical Datawarehouse.**



CREDITS

Content of the slides:

- » POD- ITBA

Images:

- » POD - ITBA
- » obtained from: commons.wikimedia.org

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by [Unsplash](https://unsplash.com)
- 